PROJECT DOCUMENTATION & IMPLEMENTATION GUIDE

STUDENT MANAGEMENT SYSTEM

A full-stack academic administration application

Technology: Python 3 • Flask • MySQL • SQLAlchemy • HTML • CSS • Bootstrap

A portfolio-ready project guide for Computer Science and Engineering students

Includes requirements, architecture, database design, implementation, testing, deployment guidance and GitHub workflow.

Version 1.0 | November 2026

Document information

Field	Details
Project name	Student Management System (SMS)
Document type	Project report, implementation guide and portfolio reference
Application type	Web-based, role-aware CRUD application
Primary stack	Python, Flask, MySQL, Flask-SQLAlchemy, Flask-Login
UI stack	Jinja2 templates, HTML5, CSS3, Bootstrap 5
Intended audience	CSE students, reviewers, mentors and entry-level developers
Version	1.0
Status	Reference implementation guide; validate locally before production use

How to use this guide: Read the design sections first, then build the application file by file. Run the tests, capture genuine screenshots from your running application, and update the README with your own setup details. Do not claim features are complete until you have run and verified them.

Project scope

This guide defines a practical Student Management System for a small college department. It focuses on maintaining student records, departments, courses, enrolments, attendance and assessment marks. The application uses a relational database and a browser-based interface.

Important implementation note

The code is a learning-oriented baseline, not a certified institutional student information system. Before real deployment, add HTTPS, production secrets management, backups, audit logging, stronger authorization rules, privacy review and security testing.

Table of contents

1. Abstract and executive summary

2. Problem statement and motivation

3. Objectives, scope and stakeholders

4. Existing system and proposed solution

5. Functional requirements

6. Non-functional requirements

7. Feasibility study and project plan

8. Technology stack

9. System architecture

10. Use cases and user journeys

11. Database design

12. Interface and navigation design

13. Project setup and environment

14. Complete implementation — configuration and models

15. Complete implementation — authentication and routes

16. Complete implementation — templates and styling

17. Validation, errors and security

18. Testing strategy and test cases

19. Running, troubleshooting and deployment

20. GitHub repository and contribution workflow

21. Screenshots and demonstration checklist

22. Future enhancements

23. Conclusion and references

Appendix A. Suggested repository structure

Appendix B. Project completion checklist

1. Abstract and executive summary

A Student Management System is a web application that helps authorized staff maintain core academic records in one place. It replaces scattered spreadsheets and paper registers with structured records that can be searched, created, updated and reviewed through a browser.

The proposed application is built with Python and Flask. MySQL stores persistent data, while Flask-SQLAlchemy maps database tables to Python model classes. Flask-Login manages user sessions. Jinja2 renders pages, and Bootstrap provides a responsive interface.

Core capabilities

- Secure sign-in and sign-out for application users.
- Create, view, edit and deactivate student records.
- Maintain departments and course information.
- Register students for courses through enrolment records.
- Record attendance and assessment marks.
- Search and filter student records.
- View dashboard counts and basic academic summaries.
- Export a student list to CSV for permitted administrative use.

Expected outcome

At completion, the repository should contain a runnable Flask application, database schema/model definitions, templates, static assets, tests, dependency list, environment example and clear setup instructions. A reviewer should be able to clone the repository, configure a local database and reproduce the demonstration.

Keywords

Student information system, Flask, Python, MySQL, CRUD, ORM, authentication, relational database, web development.

2. Problem statement and motivation

In a small institution or department, student information may be distributed across spreadsheets, paper forms and individual staff files. This creates duplicated records, inconsistent formats, slow searches and difficulty tracking enrolments or attendance.

Problem statement

Design and implement a secure, maintainable web application that centralizes essential student and academic records, reduces repetitive manual work and provides authorized users with reliable search and reporting functions.

Common issues in a manual process

- Repeated entry of the same student details in multiple files.
- Difficulty finding a record when student identifiers or names are inconsistent.
- Accidental overwriting of spreadsheet data.
- Limited traceability when records are edited.
- Attendance and marks maintained separately from student profiles.
- Reports take time to prepare and may contain outdated data.

Why this is a useful CSE portfolio project

The project demonstrates several practical skills together: relational data modelling, backend routing, form validation, authentication, CRUD operations, database migrations, testing, documentation and version control. It is more meaningful than a static page because it requires a complete data lifecycle.

3. Objectives, scope and stakeholders

Objectives

1. Build a browser-based interface for managing student records.
2. Store records in a normalized relational database.
3. Prevent unauthenticated visitors from accessing protected pages.
4. Validate user input on the server before saving it.
5. Support department, course and enrolment relationships.
6. Record attendance and assessment information with clear constraints.
7. Provide searchable views and a basic CSV export.
8. Document setup, tests and deployment considerations.

In scope

- Single institution / department deployment for a student project demonstration.
- Admin/staff user accounts created through a controlled setup process.
- Student profile, department, course, enrolment, attendance and marks records.
- Basic dashboard summaries and CSV export.

Out of scope for version 1

- Online fee payments, payroll, biometric attendance and parent portals.
- Government identity verification or integration with official education databases.
- Automated admission decisions, ranking or disciplinary workflows.
- Multi-campus tenancy, enterprise single sign-on and mobile native apps.

Stakeholders

Stakeholder	Need / responsibility
Administrator	Manage access and maintain reference data.
Faculty / staff	View student records and maintain permitted academic data.
Student (future role)	May view own profile and academic information in a later version.
Project developer	Build, test, document and maintain the application.
Reviewer / mentor	Evaluate design quality, correctness and code clarity.

4. Existing system and proposed solution

Area	Manual / spreadsheet approach	Proposed application
Storage	Separate files and paper registers	Central MySQL database
Search	Manual filtering and file browsing	Server-side search and filters
Validation	Depends on staff discipline	Required fields and database constraints
Access	Files may be copied or shared widely	Login-protected pages
Updates	Conflicting copies may exist	Single authoritative record
Reporting	Manual counting and formatting	Dashboard summaries and CSV export

Proposed workflow

1. An authorized staff member signs in.
2. Staff open the student directory and search by student number or name.
3. Staff create a new profile or edit an existing record.
4. The server validates the form and checks uniqueness constraints.
5. The ORM writes the change to MySQL.
6. The application returns a success message and refreshed record view.
7. Academic staff can add enrolments, attendance and marks according to their access rules.

> The system reduces repetitive work, but it does not automatically guarantee that entered data is correct. Institutions still need data-entry procedures, backups and access reviews.

5. Functional requirements

ID	Requirement	Priority
FR-01	The system shall allow a registered user to sign in and sign out.	Must
FR-02	The system shall prevent anonymous access to protected pages.	Must
FR-03	A user shall create a student record with a unique student number.	Must
FR-04	A user shall view a paginated or searchable student directory.	Must
FR-05	A user shall update student contact and academic details.	Must
FR-06	A user shall deactivate a student record without deleting history.	Should
FR-07	The system shall maintain departments and courses.	Must
FR-08	A student shall be associated with courses through enrolment records.	Must
FR-09	Authorized staff shall record attendance for an enrolled student.	Should
FR-10	Authorized staff shall record marks for an enrolled student.	Should
FR-11	The dashboard shall display basic record counts.	Should
FR-12	The system shall export filtered student data as CSV.	Could

Acceptance criteria

- A student number cannot be duplicated.
- Submitting an invalid email address displays a useful validation error.
- Unauthenticated requests to protected routes redirect to the login page.
- Creating a student record makes it visible in the directory after saving.
- Editing a record persists after logout and login.
- CSV export contains only the columns documented by the application.

6. Non-functional requirements

Category	Target / design decision
Usability	Consistent navigation, readable labels, responsive layout and clear feedback.
Performance	Common directory and detail pages should respond promptly on a local development setup.
Security	Password hashing, CSRF protection, server-side validation and session controls.
Reliability	Use database constraints and transactions for related writes.
Maintainability	Separate configuration, models, routes, templates and tests.
Portability	Document Python and MySQL versions; use environment variables for secrets.
Accessibility	Use semantic HTML, labels, keyboard-accessible controls and adequate contrast.
Privacy	Collect only necessary fields and restrict access to personal information.
Scalability	Use indexes on frequently searched identifiers and paginate large result sets.

Quality attributes to demonstrate

A good portfolio implementation explains trade-offs. For example, CSV export is convenient but can expose personal data; therefore, restrict it to authorized users and export only required columns. Soft deletion preserves academic history, but inactive records must be clearly marked.

7. Feasibility study and project plan

Technical feasibility

The stack is suitable for a student project. Python and Flask are approachable, MySQL is widely used for relational data, and SQLAlchemy reduces repetitive SQL while retaining the ability to inspect generated queries.

Economic feasibility

The project can be developed using free/open-source tools and a local development database. Hosting may incur a cost if a public deployment is required; a free tier can be used only after checking its current limits and terms.

Operational feasibility

The interface should be usable by staff with basic browser skills. A short user guide and consistent form validation reduce training needs.

Suggested 4-week plan

Week	Work items	Deliverable
1	Requirements, schema, wireframes, repository setup	Design and initial project skeleton
2	Authentication, student CRUD, departments and courses	Core application
3	Enrolments, attendance, marks, dashboard and export	Feature-complete prototype
4	Tests, bug fixes, screenshots, README and demo	Portfolio-ready release

Risk register

Risk	Impact	Mitigation
Database connection failure	Application cannot start	Validate environment variables and document setup.
Duplicate student identifiers	Data integrity issue	Unique database constraint plus friendly validation.
Secret committed to Git	Account compromise	Use .env locally and commit only .env.example.
Unauthorized record access	Privacy incident	Protect routes and enforce role checks.
Scope expansion	Late delivery	Freeze version 1 features and track enhancements separately.

8. Technology stack

Technology	Purpose	Why it is used
Python 3.11+	Application language	Readable, productive and widely used.
Flask	Web framework	Small core with explicit application structure.
MySQL 8+	Relational database	Stores linked academic records.
Flask-SQLAlchemy	ORM integration	Maps Python classes to database tables.
Flask-Login	Session authentication	Manages logged-in user sessions.
Flask-WTF	Forms and CSRF	Helps protect form submissions.
Jinja2	Template engine	Renders dynamic HTML safely by default.
Bootstrap 5	UI framework	Responsive layout and common components.
pytest	Testing	Automates unit and route-level checks.

Development tools

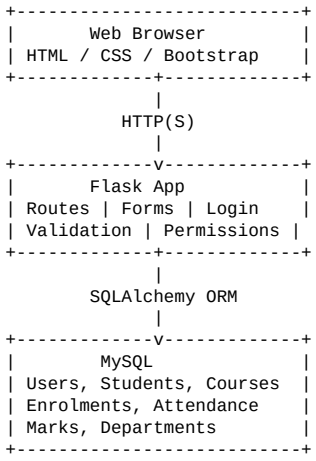
- Visual Studio Code or another Python-capable editor.
- Git and a GitHub account for version control.
- MySQL Community Server and MySQL Workbench (optional GUI).
- Python virtual environment (venv) for dependency isolation.
- Browser developer tools for debugging the interface.

Why not put everything in one Python file?

Separating the application into configuration, models, routes, templates and tests makes it easier to review, extend and debug. It also makes the GitHub repository look like a maintainable software project rather than a single classroom script.

9. System architecture

The application follows a simple three-layer web architecture. The browser sends HTTP requests to Flask routes. Routes validate input and call model/database operations. The database stores the records and returns query results.



Request lifecycle

1. Browser submits a request to a Flask endpoint.
2. Flask-Login checks whether the current session is authenticated.
3. The route reads and validates submitted data.
4. SQLAlchemy queries or modifies the relevant model.
5. MySQL enforces relational constraints and stores committed changes.
6. Flask returns a rendered template, redirect or structured error.

Design principles

- Keep business rules out of HTML templates.
- Use database constraints as a final integrity boundary.
- Use POST for state-changing operations and GET for retrieval.
- Use Post/Redirect/Get after successful form submission.
- Keep secrets and environment-specific settings outside source control.

10. Use cases and user journeys

Actor	Use case	Precondition
Staff	Sign in	Account exists and is active.
Staff	Search students	User is authenticated.
Staff	Create student	User has student-management permission.
Staff	Edit student	Student record exists.
Admin	Manage departments	User has administrator role.
Staff	Create course enrolment	Student and course exist.
Faculty	Record attendance	Student is enrolled in the course.
Faculty	Record marks	Student is enrolled in the course.
Staff	Export CSV	User has export permission.

Use case: create a student

1. Staff chooses Students → Add Student.
2. The form requests student number, full name, email, phone, department and admission year.
3. Staff submits the form.
4. The server checks required fields, email format and duplicate student number.
5. If valid, the application stores the record and redirects to the student detail page.
6. If invalid, the form is re-rendered with helpful messages.

Use case: record attendance

1. Faculty selects a course and date.
2. The application lists students enrolled in that course.
3. Faculty marks each student present or absent.
4. The server checks that each student is enrolled and that a record for the same session does not already exist.
5. The attendance records are saved and a confirmation is displayed.

> For a real institution, define exactly who can view, change or export each data category. This guide's baseline routes are not a substitute for a full institutional authorization policy.

11. Database design

Core entities

Entity	Key fields	Purpose
User	id, username, password_hash, role, is_active	Application login and role.
Department	id, name, code	Academic department.
Student	id, student_no, name, email, phone, department_id	Student profile.
Course	id, code, title, credits, department_id	Course catalogue.
Enrolment	id, student_id, course_id, academic_year, semester	Student-course relationship.
Attendance	id, enrolment_id, class_date, status	Attendance for a course session.
Mark	id, enrolment_id, assessment, score, max_score	Assessment result.

Relationship summary

- One department has many students.
- One department offers many courses.
- One student can have many enrolments.
- One course can have many enrolments.
- Enrolment resolves the many-to-many relationship between students and courses.
- One enrolment can have multiple attendance sessions and assessment marks.

Integrity constraints

- student_no must be unique and not null.
- department code and course code should be unique.
- A student-course-year-semester combination should not be duplicated.
- Attendance status should be restricted to Present, Absent or Late.
- Marks must be greater than or equal to zero and no greater than max_score.

Indexing

Add indexes to student_no, name, course code and foreign keys used in joins. Avoid adding indexes to every column: indexes consume storage and can slow writes. Measure query performance before adding more.

12. Interface and navigation design

Suggested navigation

Menu	Screen / purpose
Dashboard	Counts for students, departments and courses.
Students	Search, view, add and edit student profiles.
Departments	View and manage departments.
Courses	View and manage course catalogue.
Enrolments	Register students for courses.
Attendance	Record and review course attendance.
Marks	Record and review assessment scores.
Account	Sign out and view current account information.

Student form fields

Field	Input rule
Student number	Required, unique, trimmed; define an institution-specific format.
Full name	Required; allow spaces, hyphens and apostrophes.
Email	Optional or required according to institution policy; validate format.
Phone	Optional; store as text to preserve leading zeros and country codes.
Department	Must reference an existing department.
Admission year	A valid four-digit year within an agreed range.
Status	Active or inactive; inactive records remain available to authorized staff.

Usability guidance

- Keep labels visible; do not rely only on placeholder text.
- Show validation errors next to the relevant field.
- Ask for confirmation before destructive or irreversible actions.
- Use clear empty states such as 'No students match this search'.
- Make tables usable on narrow screens with responsive wrappers.
- Do not show sensitive fields unless the user needs them.

13. Project setup and environment

Prerequisites

- Python 3.11 or later installed.
- MySQL Server 8 or a compatible MySQL instance.
- Git installed and configured.
- A code editor and modern browser.

Create the project

```
mkdir student-management-system
cd student-management-system

python -m venv .venv

# Windows PowerShell
.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate

mkdir -p app/templates app/static/css tests
```

Install dependencies

```
pip install Flask Flask-SQLAlchemy Flask-Login Flask-WTF
pip install PyMySQL python-dotenv email-validator pytest
pip freeze > requirements.txt
```

Create a MySQL database and user

```
CREATE DATABASE student_management
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

CREATE USER 'sms_user'@'localhost'
  IDENTIFIED BY 'choose_a_strong_local_password';

GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX
  ON student_management.* TO 'sms_user'@'localhost';

FLUSH PRIVILEGES;
```

Use a unique password on your own machine. The example password is a placeholder and must not be reused. Never commit real database credentials to GitHub.

Environment file

```
# .env (keep this file out of Git)
SECRET_KEY=replace-with-a-long-random-secret
DATABASE_URL=mysql+pymysql://sms_user:YOUR_PASSWORD@localhost/student_management
FLASK_DEBUG=1
```

14. Complete implementation — configuration and models

The following files form the foundation of the reference implementation. Create each file at the path shown. Keep the names consistent with the repository structure in Appendix A.

app/__init__.py

```
import os
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager
from flask_wtf.csrf import CSRFProtect
from dotenv import load_dotenv

load_dotenv()

db = SQLAlchemy()
login_manager = LoginManager()
csrf = CSRFProtect()

def create_app():
    app = Flask(__name__)
    app.config["SECRET_KEY"] = os.getenv("SECRET_KEY", "dev-only-change-me")
    app.config["SQLALCHEMY_DATABASE_URI"] = os.getenv("DATABASE_URL")
    app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

    if not app.config["SQLALCHEMY_DATABASE_URI"]:
        raise RuntimeError("DATABASE_URL is missing. Configure your .env file.")

    db.init_app(app)
    login_manager.init_app(app)
    csrf.init_app(app)
    login_manager.login_view = "main.login"
    login_manager.login_message_category = "warning"

    from app import models
    from app.routes import main
    app.register_blueprint(main)

    with app.app_context():
        db.create_all()

    return app
```

app/models.py

```
from datetime import datetime
from werkzeug.security import generate_password_hash, check_password_hash
from flask_login import UserMixin
from app import db, login_manager

class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    password_hash = db.Column(db.String(255), nullable=False)
    role = db.Column(db.String(20), nullable=False, default="staff")
    is_active = db.Column(db.Boolean, default=True, nullable=False)

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)

@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))

class Department(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    code = db.Column(db.String(20), unique=True, nullable=False)
    name = db.Column(db.String(120), unique=True, nullable=False)
    students = db.relationship("Student", backref="department", lazy=True)
    courses = db.relationship("Course", backref="department", lazy=True)

class Student(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    student_no = db.Column(db.String(30), unique=True, nullable=False, index=True)
    full_name = db.Column(db.String(120), nullable=False, index=True)
```

```
    email = db.Column(db.String(120), unique=True)
    phone = db.Column(db.String(25))
    admission_year = db.Column(db.Integer, nullable=False)
    is_active = db.Column(db.Boolean, default=True, nullable=False)
    department_id = db.Column(db.Integer, db.ForeignKey("department.id"),
                              nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

class Course(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    code = db.Column(db.String(20), unique=True, nullable=False)
    title = db.Column(db.String(140), nullable=False)
    credits = db.Column(db.Integer, nullable=False, default=3)
    department_id = db.Column(db.Integer, db.ForeignKey("department.id"),
                              nullable=False)

class Enrolment(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    student_id = db.Column(db.Integer, db.ForeignKey("student.id"), nullable=False)
    course_id = db.Column(db.Integer, db.ForeignKey("course.id"), nullable=False)
    academic_year = db.Column(db.String(9), nullable=False)
    semester = db.Column(db.Integer, nullable=False)
    student = db.relationship("Student", backref="enrolments")
    course = db.relationship("Course", backref="enrolments")
    __table_args__ = (
        db.UniqueConstraint("student_id", "course_id", "academic_year",
                            "semester", name="uq_student_course_term"),
    )

class Attendance(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    enrolment_id = db.Column(db.Integer, db.ForeignKey("enrolment.id"),
                             nullable=False)
    class_date = db.Column(db.Date, nullable=False)
    status = db.Column(db.String(10), nullable=False)
    enrolment = db.relationship("Enrolment", backref="attendance")
    __table_args__ = (
        db.UniqueConstraint("enrolment_id", "class_date",
                            name="uq_attendance_session"),
    )

class Mark(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    enrolment_id = db.Column(db.Integer, db.ForeignKey("enrolment.id"),
                             nullable=False)
    assessment = db.Column(db.String(80), nullable=False)
    score = db.Column(db.Float, nullable=False)
    max_score = db.Column(db.Float, nullable=False)
    enrolment = db.relationship("Enrolment", backref="marks")
```

Why these models matter

The ORM classes describe the relational structure. Unique constraints prevent duplicate identifiers and duplicate course enrolments. Foreign keys keep related records connected. The password is stored as a one-way hash, never as plain text.

15. Complete implementation — authentication and routes

app/routes.py

```
import csv
import io
from datetime import datetime
from flask import (Blueprint, render_template, request, redirect, url_for,
                   flash, Response)
from flask_login import login_user, logout_user, login_required, current_user
from sqlalchemy import or_
from app import db
from app.models import User, Student, Department, Course

main = Blueprint("main", __name__)

def admin_required():
    return current_user.is_authenticated and current_user.role == "admin"

@main.route("/")
@login_required
def dashboard():
    return render_template(
        "dashboard.html",
        student_count=Student.query.count(),
        department_count=Department.query.count(),
        course_count=Course.query.count()
    )

@main.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")
        user = User.query.filter_by(username=username, is_active=True).first()
        if user and user.check_password(password):
            login_user(user)
            return redirect(url_for("main.dashboard"))
        flash("Invalid username or password.", "danger")
    return render_template("login.html")

@main.route("/logout", methods=["POST"])
@login_required
def logout():
    logout_user()
    flash("You have signed out.", "info")
    return redirect(url_for("main.login"))

@main.route("/students")
@login_required
def students():
    q = request.args.get("q", "").strip()
    query = Student.query
    if q:
        like = f"%{q}%"
        query = query.filter(or_(Student.student_no.like(like),
                                 Student.full_name.like(like),
                                 Student.email.like(like)))
    records = query.order_by(Student.full_name).all()
    return render_template("students.html", students=records, q=q)

@main.route("/students/new", methods=["GET", "POST"])
@login_required
def add_student():
    departments = Department.query.order_by(Department.name).all()
    if request.method == "POST":
        student_no = request.form.get("student_no", "").strip()
        full_name = request.form.get("full_name", "").strip()
        email = request.form.get("email", "").strip() or None
        phone = request.form.get("phone", "").strip()
        department_id = request.form.get("department_id", type=int)
        admission_year = request.form.get("admission_year", type=int)

        errors = []
        if not student_no or not full_name:
            errors.append("Student number and full name are required.")
        if not department_id or not Department.query.get(department_id):
            errors.append("Choose a valid department.")
        if not admission_year or not 2000 <= admission_year <= 2100:
            errors.append("Enter a valid admission year.")
        if Student.query.filter_by(student_no=student_no).first():
```

```
            errors.append("That student number already exists.")
        if email and Student.query.filter_by(email=email).first():
            errors.append("That email address is already in use.")

        if errors:
            for error in errors:
                flash(error, "danger")
        else:
            student = Student(student_no=student_no, full_name=full_name,
                              email=email, phone=phone,
                              department_id=department_id,
                              admission_year=admission_year)
            db.session.add(student)
            db.session.commit()
            flash("Student record created.", "success")
            return redirect(url_for("main.students"))

    return render_template("student_form.html", departments=departments,
                           student=None)

@main.route("/students/<int:student_id>/edit", methods=["GET", "POST"])
@login_required
def edit_student(student_id):
    student = Student.query.get_or_404(student_id)
    departments = Department.query.order_by(Department.name).all()
    if request.method == "POST":
        student.full_name = request.form.get("full_name", "").strip()
        student.email = request.form.get("email", "").strip() or None
        student.phone = request.form.get("phone", "").strip()
        student.department_id = request.form.get("department_id", type=int)
        db.session.commit()
        flash("Student record updated.", "success")
        return redirect(url_for("main.students"))
    return render_template("student_form.html", departments=departments,
                           student=student)

@main.route("/students/<int:student_id>/deactivate", methods=["POST"])
@login_required
def deactivate_student(student_id):
    student = Student.query.get_or_404(student_id)
    student.is_active = False
    db.session.commit()
    flash("Student marked inactive.", "info")
    return redirect(url_for("main.students"))

@main.route("/students/export.csv")
@login_required
def export_students():
    output = io.StringIO()
    writer = csv.writer(output)
    writer.writerow(["Student No", "Full Name", "Email", "Phone",
                     "Admission Year", "Department", "Status"])
    for s in Student.query.order_by(Student.full_name).all():
        writer.writerow([s.student_no, s.full_name, s.email or "", s.phone or "",
                         s.admission_year, s.department.name,
                         "Active" if s.is_active else "Inactive"])
    return Response(output.getvalue(), mimetype="text/csv",
                    headers={"Content-Disposition":
                             "attachment; filename=students.csv"})
```

This compact baseline illustrates the core student workflow. It does not yet implement full CRUD screens for departments/courses, detailed attendance/marks routes or granular permissions. Treat those as planned extensions rather than claiming they are already present.

app.py

```
from app import create_app

app = create_app()

if __name__ == "__main__":
    app.run(debug=True)
```

Create the first administrator

```
# Run once from a Python shell after configuring .env:
from app import create_app, db
from app.models import User

app = create_app()
with app.app_context():
    if not User.query.filter_by(username="admin").first():
        user = User(username="admin", role="admin")
```

```
user.set_password("change-this-password-now")
db.session.add(user)
db.session.commit()
print("Admin created. Change the password before use.")
```

16. Complete implementation — templates and styling

Create the following template files inside app/templates. The shared base template keeps the navigation consistent.

app/templates/base.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}Student Management{% endblock %}</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">
  <link href="{{ url_for('static', filename='css/style.css') }}" rel="stylesheet">
</head>
<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-primary">
  <div class="container">
    <a class="navbar-brand" href="{{ url_for('main.dashboard') }}">CampusDesk</a>
    {% if current_user.is_authenticated %}
      <div class="navbar-nav me-auto">
        <a class="nav-link" href="{{ url_for('main.students') }}">Students</a>
      </div>
      <form method="post" action="{{ url_for('main.logout') }}">
        <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
        <button class="btn btn-outline-light btn-sm">Sign out</button>
      </form>
    {% endif %}
  </div>
</nav>
<main class="container py-4">
  {% with messages = get_flashed_messages(with_categories=true) %}
    {% for category, message in messages %}
      <div class="alert alert-{{ category }}">{{ message }}</div>
    {% endfor %}
  {% endwith %}
  {% block content %}{% endblock %}
</main>
</body>
</html>
```

app/templates/login.html

```
{% extends "base.html" %}
{% block title %}Sign in | CampusDesk{% endblock %}
{% block content %}
<div class="row justify-content-center">
  <div class="col-md-5">
    <h1 class="h3 mb-3">Staff sign in</h1>
    <form method="post">
      <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
      <label class="form-label" for="username">Username</label>
      <input class="form-control mb-3" id="username" name="username"
             autocomplete="username" required>
      <label class="form-label" for="password">Password</label>
      <input class="form-control mb-3" id="password" name="password"
             type="password" autocomplete="current-password" required>
      <button class="btn btn-primary w-100">Sign in</button>
    </form>
  </div>
</div>
{% endblock %}
```

app/templates/dashboard.html

```
{% extends "base.html" %}
{% block content %}
<h1 class="h3 mb-4">Dashboard</h1>
<div class="row g-3">
  <div class="col-md-4"><div class="card card-body">
    <span class="text-muted">Students</span><strong class="display-6">{{ student_count }}</strong>
  </div></div>
  <div class="col-md-4"><div class="card card-body">
    <span class="text-muted">Departments</span><strong class="display-6">{{ department_count }}</strong>
  </div></div>
  <div class="col-md-4"><div class="card card-body">
    <span class="text-muted">Courses</span><strong class="display-6">{{ course_count }}</strong>
  </div></div>
</div>
<a class="btn btn-primary mt-4" href="{{ url_for('main.students') }}">Open students</a>
{% endblock %}
```

app/templates/students.html

```
{% extends "base.html" %}
{% block content %}
<div class="d-flex justify-content-between align-items-center mb-3">
  <h1 class="h3">Students</h1>
  <a class="btn btn-primary" href="{{ url_for('main.add_student') }}">Add student</a>
</div>
<form class="row g-2 mb-3" method="get">
  <div class="col-md-6">
    <input class="form-control" name="q" value="{{ q }}"
           placeholder="Search student number, name or email">
  </div>
  <div class="col-auto"><button class="btn btn-outline-primary">Search</button></div>
  <div class="col-auto"><a class="btn btn-outline-secondary"
       href="{{ url_for('main.export_students') }}">Export CSV</a></div>
</form>
<div class="table-responsive">
<table class="table table-striped align-middle">
  <thead><tr><th>Student No</th><th>Name</th><th>Department</th>
    <th>Status</th><th>Action</th></tr></thead>
  <tbody>
  {% for s in students %}
    <tr>
      <td>{{ s.student_no }}</td><td>{{ s.full_name }}</td>
      <td>{{ s.department.name }}</td>
      <td>{{ "Active" if s.is_active else "Inactive" }}</td>
      <td><a href="{{ url_for('main.edit_student', student_id=s.id) }}">Edit</a></td>
    </tr>
  {% else %}
    <tr><td colspan="5" class="text-center text-muted">No matching students.</td></tr>
  {% endfor %}
  </tbody>
</table>
</div>
{% endblock %}
```

app/templates/student_form.html

```
{% extends "base.html" %}
{% block content %}
<h1 class="h3">{{ "Edit" if student else "Add" }} student</h1>
<form method="post" class="row g-3">
  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
  {% if not student %}
  <div class="col-md-6">
    <label class="form-label">Student number</label>
    <input class="form-control" name="student_no" required>
  </div>
  {% endif %}
  <div class="col-md-6">
    <label class="form-label">Full name</label>
    <input class="form-control" name="full_name"
      value="{{ student.full_name if student else '' }}" required>
  </div>
  <div class="col-md-6">
    <label class="form-label">Email</label>
    <input class="form-control" type="email" name="email"
      value="{{ student.email if student and student.email else '' }}">
  </div>
  <div class="col-md-6">
    <label class="form-label">Phone</label>
    <input class="form-control" name="phone"
      value="{{ student.phone if student else '' }}">
  </div>
  <div class="col-md-6">
    <label class="form-label">Admission year</label>
    <input class="form-control" type="number" name="admission_year"
      value="{{ student.admission_year if student else '' }}" required>
  </div>
  <div class="col-md-6">
    <label class="form-label">Department</label>
    <select class="form-select" name="department_id" required>
      <option value="">Choose department</option>
      {% for d in departments %}
      <option value="{{ d.id }}" {% if student and student.department_id == d.id %}
        selected{% endif %}>{{ d.name }}</option>
      {% endfor %}
    </select>
  </div>
  <div class="col-12">
    <button class="btn btn-primary">Save</button>
    <a class="btn btn-outline-secondary" href="{{ url_for('main.students') }}">Cancel</a>
  </div>
</form>
```

```
{% endblock %}
```

17. Validation, errors and security

Input validation

Browser-side attributes such as required and type=email improve usability but can be bypassed. Always validate submitted data on the server. Trim whitespace, enforce sensible lengths, validate foreign keys and handle database uniqueness errors.

Security controls

Control	Implementation guidance
Password storage	Use Werkzeug password hashing; never store plaintext passwords.
CSRF	Enable Flask-WTF CSRFProtect and include a token in every state-changing form.
SQL injection	Use SQLAlchemy parameterized ORM operations; never concatenate user input into SQL.
XSS	Keep Jinja autoescaping enabled; do not mark untrusted content safe.
Session security	Use a strong secret, secure cookies and HTTPS in production.
Authorization	Check role and ownership on the server for every protected operation.
Secrets	Load credentials from environment variables; keep .env out of Git.
Data minimization	Store only fields needed for the stated purpose.
Backups	Schedule and test database backups; restrict backup file access.

Production configuration

```
# Example production settings to add in create_app():
app.config.update(
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SECURE=True,   # Enable only behind HTTPS
    SESSION_COOKIE_SAMESITE="Lax",
    REMEMBER_COOKIE_HTTPONLY=True,
)
```

Error handling

- Show user-friendly messages rather than raw SQL or stack traces.
- Log diagnostic details to a protected server log.
- Return 404 when a requested student/course does not exist.
- Roll back the database session after a failed transaction.
- Do not expose passwords, secret keys or database connection strings in logs.

The sample uses debug mode only for local development. Never run a public production service with Flask debug mode enabled.

18. Testing strategy and test cases

Testing levels

- Unit tests: validate small functions and model methods.
- Integration tests: verify routes, database writes and relationships.
- Security tests: verify login requirements, CSRF handling and authorization.
- Manual UI tests: check forms, navigation, responsiveness and feedback.
- Regression tests: rerun the suite after changing existing behavior.

Test case matrix

ID	Scenario	Expected result
TC-01	Open dashboard while signed out	Redirect to login.
TC-02	Sign in with valid credentials	User reaches dashboard.
TC-03	Sign in with wrong password	Error message; no session created.
TC-04	Create student with valid data	Record saved and visible in directory.
TC-05	Create duplicate student number	Validation error; no duplicate saved.
TC-06	Submit missing student name	Required-field validation shown.
TC-07	Search by student number	Matching student appears.
TC-08	Edit a student	Updated data persists.
TC-09	Export CSV	CSV downloads with expected headers.
TC-10	Submit logout request	Session ends and user returns to login.

Example test: password hashing

```
from app.models import User

def test_password_is_hashed():
    user = User(username="sample", role="staff")
    user.set_password("correct-horse-battery")
    assert user.password_hash != "correct-horse-battery"
    assert user.check_password("correct-horse-battery")
    assert not user.check_password("wrong-password")
```

Manual verification record

For each test, record the test date, application version, test data, actual result, pass/fail status and any issue ID. Include only synthetic student data in screenshots and demonstrations.

19. Running, troubleshooting and deployment

Run locally

```
# Activate your virtual environment first
python app.py
```

Open the local address printed by Flask in your browser. Create the initial admin account using the one-time shell procedure in Section 15, then sign in and add a department before creating a student.

Common problems

Problem	Likely cause	Action
DATABASE_URL missing	.env not loaded or variable absent	Check .env location and variable name.
Access denied by MySQL	Wrong username/password or grants	Check database user and privileges.
Unknown database	Schema not created	Run the CREATE DATABASE statement.
Template not found	Wrong folder or filename	Check app/templates and exact case.
CSRF token missing	Token omitted from POST form	Add csrf_token hidden input.
Duplicate key error	Identifier already exists	Choose a unique student number/email.
Module not found	Dependencies not installed	Activate venv and install requirements.

Deployment checklist

- Use a production WSGI server rather than Flask's development server.
- Set a strong SECRET_KEY and production DATABASE_URL.
- Disable debug mode.
- Use HTTPS and secure cookie settings.
- Restrict database network access and privileges.
- Configure logs, backups, monitoring and restore testing.
- Run tests and dependency vulnerability checks before release.
- Use a migration tool such as Flask-Migrate/Alembic for schema changes.

Database migrations

The example uses db.create_all() for a simple learning setup. Once a project has persistent data, do not rely on create_all() to alter existing tables. Adopt migrations so schema changes are versioned and can be applied safely.

20. GitHub repository and contribution workflow

Repository name

```
student-management-system
```

Recommended first commits

Commit message	What it records
chore: initialize Flask project structure	Folders, app factory and requirements.
feat: add database models	User, student, department and course entities.
feat: implement authentication	Login, logout and protected dashboard.
feat: add student management	Student create, list, search and edit.
feat: add CSV export	Authorized student export endpoint.
test: add model and route tests	Automated checks.
docs: write setup and usage guide	README, screenshots and project notes.

Git commands

```
git init
git add .
git commit -m "chore: initialize student management system"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/student-management-system.git
git push -u origin main
```

Good repository habits

- Commit small, meaningful changes instead of one giant final upload.
- Write commit messages that explain the change.
- Never commit .env, passwords, database dumps or real student information.
- Add screenshots only after running the actual application.
- Use Issues to track bugs and planned improvements.
- Use branches for larger features and pull requests to review changes.
- Tag stable milestones such as v1.0.0 after testing.

GitHub profile value

A repository is strongest when it demonstrates real engineering work: readable code, repeatable setup, tests, meaningful commit history, documented limitations and a working demo. A large PDF or many low-value commits do not guarantee a particular GitHub score or contribution count.

21. Screenshots and demonstration checklist

Capture screenshots from your own running application after the features have been implemented. Do not use mock screenshots as evidence of a working system.

Screenshot	What it should demonstrate
01-login.png	Sign-in screen with no real credentials visible.
02-dashboard.png	Dashboard counts using synthetic sample data.
03-student-list.png	Student directory and search.
04-add-student.png	Student form with labels and validation.
05-student-search.png	Search results for a sample student.
06-database.png	Schema view with table and relationship names; hide credentials.
07-csv-export.png	Example exported file using fictional records.
08-tests.png	Test suite output showing actual pass/fail results.

Suggested 3-minute demo

1. Introduce the problem and the intended users.
2. Sign in using a local demo account.
3. Show dashboard counts and open the student directory.
4. Search for a synthetic student record.
5. Create a new sample student and show the saved result.
6. Edit the record and demonstrate that the change persists.
7. Export the directory to CSV.
8. Briefly explain the database relationships and security controls.
9. Finish by showing the repository README and tests.

22. Future enhancements

Enhancement	Description	Learning outcome
Role-based permissions	Separate admin, faculty and read-only roles with route-level checks.	Authorization design
Full department/course CRUD	Manage reference data from the browser.	CRUD and validation
Attendance module	Session-wise attendance with percentage reports.	Relational queries
Marks module	Assessment entry, grade calculation and transcript view.	Business rules
Pagination	Load large student lists in pages.	Query efficiency
Audit log	Record who changed what and when.	Traceability
REST API	Expose documented JSON endpoints.	API design
Automated deployment	CI tests and controlled deployment pipeline.	DevOps
Docker	Containerize Flask and MySQL for repeatable setup.	Containerization
Accessibility improvements	Keyboard support and accessibility testing.	Inclusive design

Suggested version roadmap

- v1.0: Authentication, student directory, create/edit, search and CSV export.
- v1.1: Department/course management and pagination.
- v1.2: Enrolment, attendance and marks workflows.
- v2.0: Role-based authorization, audit log, REST API and deployment automation.

23. Conclusion and references

The Student Management System is a practical full-stack project that combines backend development, database design, user interface work, authentication, validation, testing and GitHub collaboration. Its value comes from a working, well-documented implementation and evidence that the developer understands the design decisions.

Build the application incrementally. Verify every feature locally, use synthetic data, test error cases and document limitations honestly. A smaller reliable application is more impressive than a long feature list that cannot be demonstrated.

References and learning resources

- Python documentation: https://docs.python.org/3/
- Flask documentation: https://flask.palletsprojects.com/
- Flask-SQLAlchemy documentation: https://flask-sqlalchemy.palletsprojects.com/
- Flask-Login documentation: https://flask-login.readthedocs.io/
- Flask-WTF documentation: https://flask-wtf.readthedocs.io/
- MySQL documentation: https://dev.mysql.com/doc/
- SQLAlchemy documentation: https://docs.sqlalchemy.org/
- Bootstrap documentation: https://getbootstrap.com/docs/
- pytest documentation: https://docs.pytest.org/

> Always check the official documentation for the versions you install. Framework APIs and security recommendations can change.

Appendix A. Suggested repository structure

```
student-management-system/
■■■ app/
■   ■■■ __init__.py
■   ■■■ models.py
■   ■■■ routes.py
■   ■■■ templates/
■   ■   ■■■ base.html
■   ■   ■■■ login.html
■   ■   ■■■ dashboard.html
■   ■   ■■■ students.html
■   ■   ■■■ student_form.html
■   ■■■ static/
■       ■■■ css/
■           ■■■ style.css
■■■ tests/
■   ■■■ __init__.py
■   ■■■ test_models.py
■■■ .env.example
■■■ .gitignore
■■■ app.py
■■■ requirements.txt
■■■ README.md
```

Example .gitignore

```
.venv/
__pycache__/
*.py[cod]
.env
.pytest_cache/
.coverage
instance/
*.log
.DS_Store
```

Example .env.example

```
SECRET_KEY=replace-me
DATABASE_URL=mysql+pymysql://username:password@localhost/student_management
FLASK_DEBUG=0
```

Example style.css

```
body {
  background: #f6f8fb;
  color: #1d2935;
}
.card {
  border: 0;
  border-radius: 0.8rem;
  box-shadow: 0 0.2rem 0.8rem rgba(20, 40, 60, 0.06);
}
.table thead th {
  white-space: nowrap;
}
```

Appendix B. Project completion checklist

■ Create the repository and folder structure.

■ Install dependencies in a virtual environment.

■ Create the MySQL database and restricted local user.

■ Configure .env and confirm it is ignored by Git.

■ Implement and verify database models.

■ Create the first administrator with a strong password.

■ Run the application and sign in successfully.

■ Add at least one department and course.

■ Create, search and edit synthetic student records.

■ Verify duplicate student numbers are rejected.

■ Verify anonymous users cannot access protected pages.

■ Verify CSV export works and contains expected fields.

■ Add tests and record actual results.

■ Run the app from a clean environment using README instructions.

■ Capture genuine screenshots with fictional data.

■ Push code to GitHub and review the repository for secrets.

■ Write known limitations and next steps in the README.

■ Tag a tested release.

Final self-review

Before presenting this project, be prepared to explain: why a relational database was selected; how the student-course many-to-many relationship is modelled; how passwords are protected; how duplicate records are prevented; how a route is protected; how you tested the application; and what you would improve next.

End of Project 1 — Student Management System